



B.TECH II YEAR I SEM --- BACKEND WEB TECHNOLOGIES

(Short Question and Answers)

UNIT-2

1. Define SQLite. Mention any two features of SQLite.

Ans:

SQLite is a highly **efficient**, **serverless**, and **self-contained** SQL database engine that stands out for its simplicity and ease of integration. Designed to be embedded within applications, **SQLite** eliminates the need for separate database server processes and complex configurations.

Features:

- **Lightweight:** SQLite is a very lightweight database engine, with a small library size (typically less than 1MB). This makes it well-suited for use in applications where the database is embedded directly into the application binary, such as **mobile apps**.
- **Serverless:** As mentioned earlier, SQLite is a serverless database engine, which means there is no need to set up and maintain a separate database server process. This makes it easy to deploy and manage, as there are no additional dependencies to worry about.
- **Cross-platform:** SQLite is available on many platforms, including **Linux, macOS, and Windows**, making it a good choice for **cross-platform** development.
- **Standalone:** SQLite stores all of the data in a single file on the filesystem, which makes it easy to copy or backup the database.

2. What is a Database Schema?

Ans:

A **Database Schema** is simply the **blueprint or structure of a database**. It defines the structure and organization of data within a database. It outlines how data is logically stored, including the relationships between different tables and other database objects. The schema serves as a blueprint for how data is stored, accessed, and manipulated, ensuring consistency and integrity throughout the system. It defines:

- What tables exist
- What columns each table has
- The data type of each column
- Primary keys
- Foreign keys
- Constraints (NOT NULL, UNIQUE, etc.)

3.Expand CRUD and list its operations.

CRUD	SQL Command	Node.js Method	Purpose
Create	INSERT	.run()	Adds or inserts new data records
Read	SELECT	.all() / .get()	Retrieves records
Update	UPDATE	.run()	Modifies existing records
Delete	DELETE	.run()	Removes records

4. What is a RESTful API?

A RESTful API (Representational State Transfer API) is a way for applications to communicate with each other over the internet using HTTP methods like GET, POST, PUT, and DELETE.

A **RESTful API** is a set of URLs (called **endpoints**) that allow clients such as browsers, mobile apps, or Postman to perform CRUD operations on a database using standard HTTP methods. In a typical Node.js application:

- **Express.js** receives the HTTP request.
- **SQLite** stores and retrieves the data.
- **Express.js** sends the response back as **JSON**.

5.Differentiate between Query Parameters and Route Parameters.

Feature	Route Parameters (Path Params)	Query Parameters (Query Strings)
URL Syntax	/users/123	/users?role=admin
Express Object	req.params	req.query
Express Definition	Defined with colons (/users/:id)	Not defined in the route path
Requirement	Strict. Essential for route matching.	Flexible. Fully optional.
Primary Use	Locating a specific entity/ resource	Sorting, filtering, searching, paginating

6. What is the purpose of a Primary Key in a database?

The PRIMARY KEY constraint uniquely identifies each record in a database table. A PRIMARY KEY constraint ensures unique values, and cannot contain NULL values (it is a combination of both a UNIQUE constraint and a NOT NULL constraint).

The primary purpose of a primary key is to uniquely identify each row (record) within a database table. It serves as a digital fingerprint, ensuring that no two rows are identical and providing a reliable way to target specific data.

LONG QUESTION AND ANSWERS:

1.Explain SQLite integration in Node.js with suitable code examples.

SQLite is a serverless relational database. When you integrate SQLite with Node.js, your Node.js application can create databases, create tables, insert data, update data, delete data, and retrieve data using SQL queries.

Step 1: Create a Project

Create a folder: SQLiteNode

Open it in VS Code then open Terminal and Initialize Node.js project.

npm init -y

This creates

package.json

Step 2: Install better-sqlite3

npm install better-sqlite3

After installation your folder will look like this

SQLiteNode

```
|  
├── node_modules  
├── package.json  
├── package-lock.json  
└── app.js
```

Step 3: Create app.js file and Database File

Don't create database.db manually.

SQLite will create it automatically.

Step 4: Open app.js and write the below code

```
const Database = require("better-sqlite3"); // Imports SQLite package.  
const db = new Database("database.db"); // Creates database if it doesn't exist  
  
module.exports = db; //Exports the database object so other files can use it.
```

Now you'll see database.db inside your folder.

SQLite created it automatically.

Now your project should look like:

SQLite-Node

```
|  
├── node_modules  
├── package.json  
├── package-lock.json  
├── database.db  
└── app.js
```

Step 5: Create a Table

Add these lines in your code

```
db.prepare(  
  CREATE TABLE IF NOT EXISTS Student(  
    id INTEGER PRIMARY KEY,  
    name TEXT,  
    branch TEXT )  
`).run();  
  
console.log("Table Created");
```

When u run above code in terminal node app.js

Output: Table Created

Step 6: Insert Data

Now add these line after your above code

```
db.prepare(  
  INSERT INTO Student(name, branch) VALUES (?, ?)  
`).run("John", "CSE");  
  
console.log("Student Added");
```

Run: node app.js

Output: Student Added

Instead of writing VALUES ('John','CSE')

we write VALUES (?,?)

Then pass values .run("John","CSE")

This helps prevent SQL injection and makes it easy to reuse the statement with different values.

Step 7: Read Data

```
const rows = db.prepare("SELECT * FROM Student").all();  
console.log(rows);
```

Output:

```
[  
  { id: 1, name: 'John', branch: 'CSE' }  
]
```

.all() means Return all rows.

Step 8: Insert Another Student

Remove John and CSE and place with Alice and ECE

And again run: node app.js

Output:

```
[  
  { id:1, name:'John', branch:'CSE'},  
  { id:2, name:'Alice', branch:'ECE'}  
]
```

2. Describe CRUD operations in Node.js using SQLite with examples.

CRUD Operations in SQLite with Node.js

CRUD stands for the four basic operations performed on data in a database:

- **C** → Create
- **R** → Read
- **U** → Update
- **D** → Delete

These operations are the foundation of almost every database-driven application.

1. Create a table

```
db.prepare(`  
CREATE TABLE IF NOT EXISTS Student(  
  id INTEGER PRIMARY KEY,  
  name TEXT,
```

```
branch TEXT
)
`).run();
```

Then insert the data/values into table

SQL Statement

```
INSERT INTO Student(name, branch)
VALUES (?, ?);
```

Node.js Code

```
db.prepare(`
INSERT INTO Student(name, branch)
VALUES (?, ?)
`).run("John", "CSE");
```

Result

Before:

id	name	branch
----	------	--------

After:

id	name	branch
1	John	CSE

2. Read (Select)

The **Read** operation retrieves data from the database.

Read All Records

```
const rows = db.prepare(`
SELECT * FROM Student
`).all();
console.log(rows);
```

Output

```
[
  { id: 1, name: "John", branch: "CSE" }
]
```

Read One Record

```
const student = db.prepare(`
SELECT * FROM Student
WHERE id = ?
`).get(1);
console.log(student);
```

Output:

```
{
  id: 1,
  name: "John",
  branch: "CSE"
}
```

Difference

Method	Returns
.all()	All matching rows as an array
.get()	Only the first matching row as an object

3. Update

The **Update** operation changes existing data.

SQL Statement

```
UPDATE Student
SET branch = ?
WHERE id = ?;
```

Node.js Code

```
db.prepare(`
UPDATE Student
SET branch = ?
WHERE id = ?
`).run("AIML", 1);
```

Result

Before:

id	name	branch
1	John	CSE

After:

id	name	branch
1	John	AIML

4. Delete

The **Delete** operation removes records from the table.

Delete One Record

```
db.prepare(`
DELETE FROM Student
WHERE id = ?
`).run(1);
```

Result:

Before:

id	name	branch
1	John	AIML

After:

The table is empty.

Delete All Records:

```
db.prepare(`
DELETE FROM Student
`).run();
```

This removes **all rows**, but the **Student** table still exists.

CRUD	SQL Command	Node.js Method	Purpose
Create	INSERT	.run()	Adds new records
Read	SELECT	.all() / .get()	Retrieves records
Update	UPDATE	.run()	Modifies existing records
Delete	DELETE	.run()	Removes records

Complete example:

```
const Database = require("better-sqlite3");
const db = new Database("database.db");
// Export (optional if using one file)
module.exports = db;

// =====
// CREATE TABLE
// =====

db.prepare(`
CREATE TABLE IF NOT EXISTS Student(
  id INTEGER PRIMARY KEY,
  name TEXT,
  branch TEXT
)
`).run();
console.log("Table Created");

// =====
// CREATE (INSERT)
// =====

db.prepare(`
INSERT INTO Student(name, branch)
VALUES (?, ?)
`).run("John", "CSE");
console.log("Student Added");

// =====
// READ (SELECT ALL)
// =====

const rows = db.prepare(`
SELECT * FROM Student
`).all();
console.log("\nAll Students:");
console.log(rows);

// =====
// READ (SELECT ONE)
// =====

const student = db.prepare(`
SELECT * FROM Student
```

```
WHERE id = ?
`).get(1);
console.log("\nStudent with ID = 1");
console.log(student);
// =====
// UPDATE
// =====
db.prepare(`
UPDATE Student
SET branch = ?
WHERE id = ?
`).run("AIML", 1);
console.log("\nStudent Updated");
// Check updated record
const updatedStudent = db.prepare(`
SELECT * FROM Student
WHERE id = ?
`).get(1);
console.log(updatedStudent);
// =====
// DELETE
// =====
db.prepare(`
DELETE FROM Student
WHERE id = ?
`).run(1);
console.log("\nStudent Deleted");
// =====
// READ AFTER DELETE
// =====
const finalRows = db.prepare(`
SELECT * FROM Student
`).all();
console.log("\nRemaining Students:");
console.log(finalRows);
```

Output:

```
PS D:\OneDrive\Desktop\SQLITE> node app.js
Table Created
Student Added

All Students:
[ { id: 1, name: 'John', branch: 'CSE' } ]

Student with ID = 1
{ id: 1, name: 'John', branch: 'CSE' }

Student Updated
{ id: 1, name: 'John', branch: 'AIML' }

Student Deleted

Remaining Students:
[]
PS D:\OneDrive\Desktop\SQLITE> █
```

3. Explain Query Writing and Query Optimization techniques in Node.js.

Query Writing is the process of writing SQL statements to retrieve, insert, update, or delete data from a database efficiently.

Query Optimization is the process of improving SQL queries so they execute faster, use fewer resources, and return results efficiently, especially when working with large databases.

For example:

- A table has 10 records → Performance difference is not noticeable.
- A table has 10 million records → Optimized queries become essential.

Query Writing and Optimization Techniques

Technique	Description	Example
Use SELECT Specific Columns	Retrieve only the columns you need instead of all columns.	SELECT name, branch FROM Student;
Use WHERE Clause	Filter records instead of retrieving the entire table.	SELECT * FROM Student WHERE branch = 'CSE';
Use Parameterized Queries	Prevent SQL injection and improve query safety.	SELECT * FROM Student WHERE id = ?;
Use LIMIT	Retrieve only the required number of records.	SELECT * FROM Student LIMIT 10;
Use ORDER BY Carefully	Sort only when required because sorting large datasets takes time.	SELECT * FROM Student ORDER BY name ASC;
Create Indexes	Speed up searches on frequently searched columns.	CREATE INDEX idx_branch ON Student(branch);
Avoid Unnecessary	LIKE '%text%' is slower than LIKE 'text%'	SELECT * FROM Student

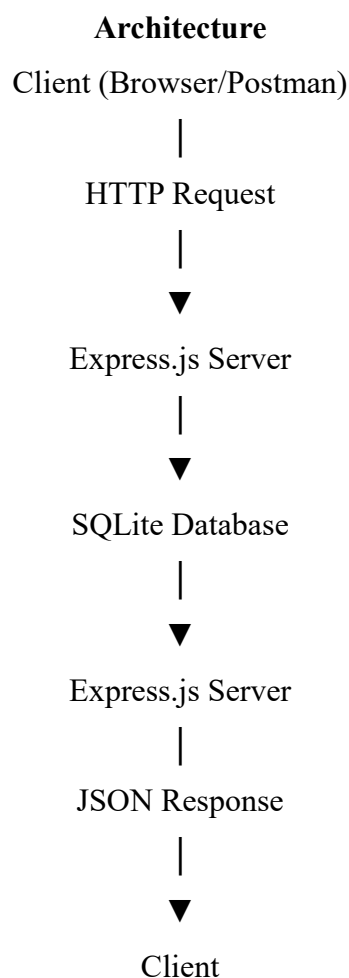
Wildcards	because it cannot efficiently use indexes.	WHERE name LIKE 'Jo%';
Use Pagination	Retrieve data page by page instead of loading all rows.	SELECT * FROM Student LIMIT 10 OFFSET 20;

4. Explain RESTful API Development in Node.js using Express.js with an example.

A **RESTful API** is a set of URLs (called **endpoints**) that allow clients such as browsers, mobile apps, or Postman to perform CRUD operations on a database using standard HTTP methods. In a typical Node.js application:

- **Express.js** receives the HTTP request.
- **SQLite** stores and retrieves the data.
- **Express.js** sends the response back as **JSON**.

In Node.js, we commonly build RESTful APIs using **Express.js**, and we connect them to a database like **SQLite**.



HTTP Methods Used in REST APIs

HTTP Method	Purpose	CRUD Operation
GET	Retrieve data	Read

POST	Add/ Insert new data	Create
PUT	Update existing data	Update
DELETE	Remove data	Delete

- **POST** /students inserts a student into the SQLite database.
- **GET** /students retrieves all students from the database.

Complete Code for CRUD Operations using HTTP Methods:

```

const express = require("express");
const Database = require("better-sqlite3");

const app = express();
const db = new Database("database.db");

// Middleware
app.use(express.json());

// Create table
db.prepare(`
CREATE TABLE IF NOT EXISTS Student(
  id INTEGER PRIMARY KEY,
  name TEXT NOT NULL,
  branch TEXT NOT NULL
)
`).run();

// =====
// POST - Insert Student
// =====
app.post("/students", (req, res) => {

  // Get data from request body
  const { id, name, branch } = req.body;

  // Insert into database
  const query = db.prepare(`

```

```

    INSERT INTO Student (id, name, branch)
    VALUES (?, ?, ?)
  `);

  query.run(id, name, branch);

  // Send response
  res.json({
    message: "Student inserted successfully!"
  });
});

// =====
// GET - Read All Students
// =====
app.get("/students", (req, res) => {

  // Fetch all students
  const students = db.prepare("SELECT * FROM Student").all();

  // Send data
  res.json(students);
});

// =====
// PUT - Update data
// =====
app.put("/students/:id", (req, res) => {

  const { branch } = req.body;

  db.prepare(
    "UPDATE Student SET branch=? WHERE id=?"
  ).run(branch, req.params.id);

  res.json({
    message: "Student Updated"
  });
});

```

```

});

});

// =====
// DELETE - Delete data
// =====

app.delete("/students/:id", (req, res) => {

  db.prepare(
    "DELETE FROM Student WHERE id=?"
  ).run(req.params.id);

  res.json({
    message: "Student Deleted"
  });

});

// Start Server
app.listen(3030, () => {
  console.log("Server running on http://localhost:3030");
});

```

Output:

For POST: <http://localhost:3030/students>

JSON:

```

{
  "id": 1,
  "name": "John"
  "branch": "CSE"
}

```

Response:

```

{
  "message": "Student inserted successfully!"
}

```

For GET: <http://localhost:3030/students>

Response:

```
[{"id":1,"name":"husna","branch":"cse"},
{"id":2,"name":"Alice","branch":"ECE"},
{"id":3,"name":"Bob","branch":"IT"},
{"id":4,"name":"john","branch":"cse"}]
```

For PUT: <http://localhost:3030/students/1>

JSON:

```
{
  "branch":"Robotics"
}
```

Response

```
{
  "message":"Student Updated"
}
```

For DELETE: <http://localhost:3030/students/4>

Response:

```
{
  "message":"Student Deleted"
}
```

5.Discuss Query Handling in Express.js using query parameters?

Query handling is the process of receiving, reading, validating, and processing query parameters sent by the client in the URL.

Query parameters are optional values added after the ? symbol in a URL. They are commonly used for searching, filtering, sorting, and pagination.

Query Handling in Express.js: Searching, Filtering, Sorting, and Pagination

These are the four most common operations performed using **query parameters (req.query)** in Express.js.

Assume the following Student table contains:

id	name	branch
1	John	CSE

2	Alice	ECE
3	David	CSE
4	Sara	IT
5	Bob	ECE

1. Searching

Searching means finding records that match a keyword or partial text.

For example, if a user searches for "Jo", it should return John.

URL: GET /students/search?name=Jo

Express Code

```
app.get("/students/search", (req, res) => {  
  
  const { name } = req.query;  
  
  const students = db  
    .prepare("SELECT * FROM Student WHERE name LIKE ?")  
    .all(`%${name}%`);  
  
  res.json(students);  
  
});
```

SQL Query

```
SELECT * FROM Student  
WHERE name LIKE '%Jo%';
```

Output

```
[  
  {  
    "id":1,  
    "name":"John",  
    "branch":"CSE"  
  }  
]
```

When to Use

- Search student by name
- Search employee by name
- Search product by title

2. Filtering

Filtering means displaying only the records that satisfy a specific condition.

For example, show only students from the CSE branch.

URL: GET /students/filter?branch=CSE

Express Code

```
app.get("/students/filter", (req, res) => {  
  
  const { branch } = req.query;  
  
  const students = db  
    .prepare("SELECT * FROM Student WHERE branch = ?")  
    .all(branch);  
  
  res.json(students);  
  
});
```

SQL Query

```
SELECT * FROM Student  
WHERE branch = 'CSE';
```

Output

```
[  
  {  
    "id":1,  
    "name":"John",  
    "branch":"CSE"  
  },  
  {  
    "id":3,
```

```
"name":"David",  
"branch":"CSE"  
}  
]
```

When to Use

- Students by branch
- Employees by department
- Products by category

3. Sorting: Sorting means arranging records in a particular order.

The order can be:

- Ascending (A–Z, 1–9)
- Descending (Z–A, 9–1)

URL: GET /students/sort?sortBy=name&order=ASC

Express Code

```
app.get("/students/sort", (req, res) => {  
  
  const sortBy = req.query.sortBy || "id";  
  const order = (req.query.order || "ASC").toUpperCase();  
  
  const sql = `SELECT * FROM Student ORDER BY ${sortBy} ${order}`;  
  
  const students = db.prepare(sql).all();  
  
  res.json(students);  
  
});
```

SQL Query

```
SELECT * FROM Student  
ORDER BY name ASC;
```

Output

Alice

Bob
David
John
Sara

Descending Order

GET /students/sort?sortBy=name&order=DESC

Output

Sara
John
David
Bob
Alice

When to Use

- Sort students by name
- Sort employees by salary
- Sort products by price

Important: In a real application, always validate sortBy and order before using them in the SQL query to avoid SQL injection.

4. Pagination

Pagination means displaying a small number of records at a time instead of returning all records.

Suppose your **student** table has 6 records.

limit means **how many records should be returned**.

If limit = 2 then the API will return **only 2 students**.

page means **which group of records you want**.

If each page has 2 records:

- Page 1 → Students 1 and 2
- Page 2 → Students 3 and 4
- Page 3 → Students 5 and 6

SQLite doesn't understand "page". It only understands:

- How many rows to return (LIMIT)
- How many rows to skip (OFFSET)

URL: GET /students?page=2&limit=2

Express Code

```
app.get("/students", (req, res) => {  
  
  const page = Number(req.query.page) || 2;  
  const limit = Number(req.query.limit) || 2;  
  
  const offset = (page - 1) * limit;  
  
  const students = db  
    .prepare("SELECT * FROM Student LIMIT ? OFFSET ?")  
    .all(limit, offset);  
  
  res.json(students);  
  
});
```

SQL Query

```
SELECT * FROM Student  
LIMIT 2 OFFSET 2;
```

Output

```
[  
  {  
    "id":3,  
    "name":"David",  
    "branch":"CSE"  
  },  
  {  
    "id":4,  
    "name":"Sara",  
    "branch":"IT"  
  }  
]
```

6. Explain the REST Principles in detail with suitable examples.

REST (Representational State Transfer) is an architectural style for designing networked applications, introduced by Roy Fielding in his 2000 doctoral dissertation. It defines a set of constraints that, when followed, make web APIs scalable, simple, and reliable.

The Six Guiding Principles:

1. Client-Server Architecture: The client and the server are separate entities, each with its own responsibilities.

- The client is responsible for the user interface and sending requests.
- The server is responsible for processing requests, executing business logic, interacting with the database, and sending responses.

This separation makes it easier to develop, maintain, and update both the client and server independently.

Example: Consider an online shopping website. A customer searches for a laptop using the website. The browser sends the search request to the server. The server searches the database for matching laptops. The server returns the list of available laptops to the browser. The browser only displays the information. It does not know how the server searched the database.

2. Statelessness: REST APIs are stateless, meaning the server does not remember information from previous requests. Every request must contain all the information required for the server to process it. The server treats every request as a completely new request.

Example: Suppose a student wants to view his profile.

- In the first request, the student asks for profile details.
- In the second request, the student asks for examination results.

The server does not remember the first request. The second request must again include all the necessary information (such as the student's ID or authentication token) so the server can identify the student. This makes REST APIs scalable because the server does not have to store client session information.

3. Cacheable: REST responses should specify whether they can be stored in a cache. Caching allows clients to reuse previously received data instead of requesting the same information repeatedly from the server. This reduces network traffic, decreases server load, and improves application performance.

Example: A university website displays the academic calendar. Since the calendar changes only occasionally, the browser can store (cache) the page for a certain period. If the student visits the page again within that period, the browser displays the cached version instead of requesting it from the server again.

4. Uniform Interface: A REST API should provide a consistent and standardized way for clients to access resources. The same rules should be followed throughout the API. Resources should be identified using meaningful URLs, and standard HTTP methods should be used to perform operations.

Example: in an online shopping application:

- /products → List of all products
- /products/101 → Details of product 101

Notice that the resource remains “product” only the http methods change to do required actions.

5. Layered System: A REST system can consist of multiple layers between the client and the server. The client does not know how many layers exist or what operations they perform.

These layers may include:

- Authentication
- Security
- Load balancing
- Logging
- Caching
- Database services

Each layer performs a specific task without affecting the client.

Example: When a user logs into an online banking application:

1. The request first passes through a security layer that verifies the user's identity.
2. Next, it passes through a firewall for protection.
3. Then it reaches the application server.
4. Finally, the application server accesses the database and retrieves the account information.

The user is unaware of these intermediate layers and simply receives the requested information.

6. Code on Demand (Optional): REST allows the server to send executable code to the client when necessary. This principle is optional and is not commonly used in REST APIs. The downloaded code can add extra functionality to the client application.

Example: A website may send JavaScript files to the user's browser to perform some interactive animations without requiring another request to the server.